

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Artificial Intelligence Application Development and Jira

Task -1

Roll No.: T005	Name: Ritesh Chaubey
Class:TYIT	Batch:1
Date of Assignment: 22/07/2026	Date/Time of Submission: 22/07/2026

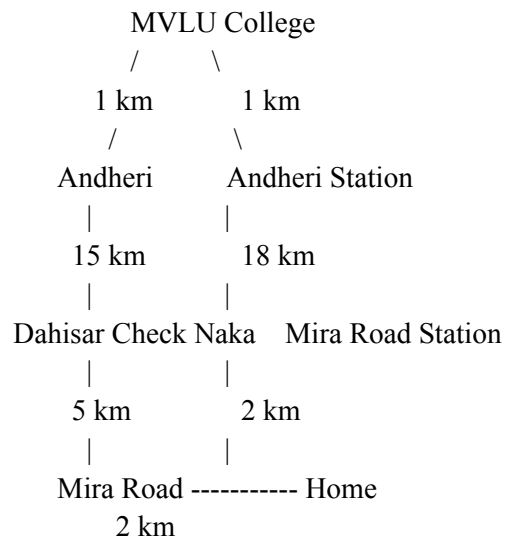
Problem Statement:-

Write Python programs to determine the route from your College to your Home using the following search algorithms:

- Breadth-First Search (BFS)
- Depth-First Search (DFS)
- Uniform Cost Search (UCS)
- Greedy Best-First Search
- A* Search

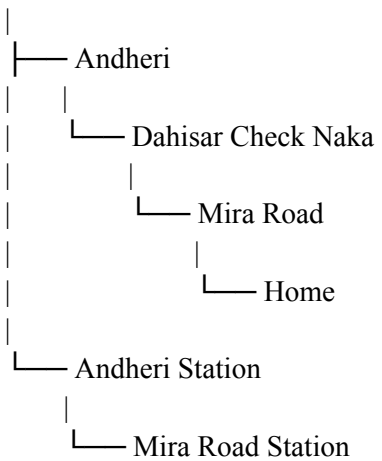
Include the graph used in your implementation along with edge costs and heuristic values if required.

Graph:-



1)BFS Traversal Graph

MVLU College



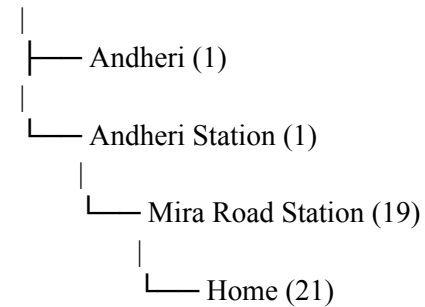
2)DFS Traversal Graph

MVLU College



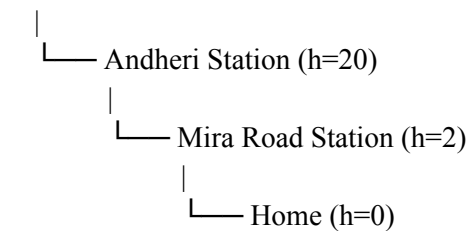
3)UCS Traversal Graph

MVLU College (0)



4)Greedy Best First Search Traversal Graph

MVLU College (h=21)



5)A* Search Traversal Graph

MVLU College

(g=0,h=21,f=21)

|
└─ Andheri Station
(g=1,h=20,f=21)

|
└─ Mira Road Station
(g=19,h=2,f=21)

|
└─ Home
(g=21,h=0,f=21)

Edge Cost Table:-

From	To	Edge Cost (km)
MVLU College	Andheri	1
MVLU College	Andheri Station	1
Andheri	Dahisar Check Naka	15
Dahisar Check Naka	Mira Road	5
Mira Road	Home	2
Andheri Station	Mira Road Station	18
Mira Road Station	Home	2

Heuristic Table (for Greedy and A*):-

Node	Heuristic Value h(n)	Reason
MVLU College	21	Estimated shortest distance to Home
Andheri	22	15 + 5 + 2
Andheri Station	20	18 + 2
Dahisar Check Naka	7	5 + 2

Mira Road	2	2 km to Home
Mira Road Station	2	2 km to Home
Home	0	Goal node

Brief Description of Each Algorithm:-

1. Breadth First Search (BFS)

BFS explores the graph level by level using a queue (FIFO).
It visits all neighboring nodes before moving to the next level.
It finds the path to the goal and returns the total cost and explored nodes.

2. Depth First Search (DFS)

DFS explores the graph by going deep into one path before backtracking.
It uses a stack (LIFO) to store nodes during the search process.
It finds a path to the goal but does not always provide the shortest path.

3. Uniform Cost Search (UCS)

UCS selects and expands the node with the lowest path cost first.
It uses a priority queue to compare the total cost of different paths.
It always finds the minimum-cost path in a weighted graph.

4. Greedy Best-First Search

Greedy Best-First Search uses the heuristic value $h(n)$ to guide the search.
It selects the node that appears closest to the goal.
It is faster but may not always find the optimal path.

5. A Search*

A* Search combines the actual cost $g(n)$ and heuristic cost $h(n)$.
It uses the formula $f(n) = g(n) + h(n)$ to select the best node.
It efficiently finds the optimal path when the heuristic is correct.

Code:-

```
from collections import deque
import heapq
```

```
# -----
# Graph (Bidirectional)
# -----
graph = {
    "MVLU College": [
        ("Andheri", 1),
        ("Andheri Station", 1)
    ],
    "Andheri": [
        ("MVLU College", 1),
        ("Dahisar Check Naka", 15)
    ],
    "Andheri Station": [
        ("MVLU College", 1),
        ("Mira Road Station", 18)
    ],
    "Dahisar Check Naka": [
        ("Andheri", 15),
        ("Mira Road", 5)
    ],
    "Mira Road": [
        ("Dahisar Check Naka", 5),
        ("Home", 2)
    ],
    "Mira Road Station": [
        ("Andheri Station", 18),
        ("Home", 2)
    ],
    "Home": [
        ("Mira Road", 2),
        ("Mira Road Station", 2)
    ]
}
# -----
# Heuristic Values (towards Home)
# -----
heuristic = {
    "MVLU College": 21,
    "Andheri": 22,
    "Andheri Station": 20,
    "Dahisar Check Naka": 7,
    "Mira Road": 2,
    "Mira Road Station": 2,
    "Home": 0
}
# -----
# Breadth First Search (BFS)
# -----
def bfs(start, goal):
```

```
queue = deque([(start, [start], 0)])
visited = {start}
explored = 0
while queue:
    node, path, cost = queue.popleft()
    explored += 1
    if node == goal:
        return path, cost, explored
    for neighbour, edge_cost in graph[node]:
        if neighbour not in visited:
            visited.add(neighbour)
            queue.append(
                (
                    neighbour,
                    path + [neighbour],
                    cost + edge_cost
                )
            )

return None
# -----
# Depth First Search (DFS)
# -----
def dfs(start, goal):
    stack = [(start, [start], 0)]
    visited = set()
    explored = 0
    while stack:
        node, path, cost = stack.pop()
        if node not in visited:
            visited.add(node)
            explored += 1
            if node == goal:
                return path, cost, explored
            for neighbour, edge_cost in reversed(graph[node]):
                if neighbour not in visited:
                    stack.append(
                        (
                            neighbour,
                            path + [neighbour],
                            cost + edge_cost
                        )
                    )

    return None
# -----
# Uniform Cost Search (UCS)
# -----
def ucs(start, goal):
```

```
    pq = [(0, start, [start])]
    visited = set()
    explored = 0
    while pq:
        cost, node, path = heapq.heappop(pq)
        if node in visited:
            continue
        visited.add(node)
        explored += 1
        if node == goal:
            return path, cost, explored
        for neighbour, edge_cost in graph[node]:
            if neighbour not in visited:
                heapq.heappush(
                    pq,
                    (
                        cost + edge_cost,
                        neighbour,
                        path + [neighbour]
                    )
                )

    return None

# -----
# Greedy Best First Search
# -----
def greedy(start, goal):
    pq = [
        (
            heuristic[start],
            start,
            [start],
            0
        )
    ]
    visited = set()
    explored = 0
    while pq:
        h, node, path, cost = heapq.heappop(pq)
        if node in visited:
            continue
        visited.add(node)
        explored += 1
        if node == goal:
            return path, cost, explored
        for neighbour, edge_cost in graph[node]:
            if neighbour not in visited:
                heapq.heappush(
                    pq,
```

```

        (
            heuristic[neighbour],
            neighbour,
            path + [neighbour],
            cost + edge_cost
        )
    )
    return None
# -----
# A* Search
# -----
def astar(start, goal):
    pq = [
        (
            heuristic[start],
            0,
            start,
            [start]
        )
    ]
    visited = set()
    explored = 0
    while pq:
        f, g, node, path = heapq.heappop(pq)
        if node in visited:
            continue
        visited.add(node)
        explored += 1
        if node == goal:
            return path, g, explored
        for neighbour, edge_cost in graph[node]:
            if neighbour not in visited:
                new_g = g + edge_cost
                new_f = new_g + heuristic[neighbour]
                heapq.heappush(
                    pq,
                    (
                        new_f,
                        new_g,
                        neighbour,
                        path + [neighbour]
                    )
                )
    return None
# -----
# Execute All Algorithms
# -----
algorithms = [
    ("Breadth First Search (BFS)", bfs),

```



```
("Depth First Search (DFS)", dfs),
("Uniform Cost Search (UCS)", ucs),
("Greedy Best First Search", greedy),
("A* Search", astar)
]
start = "MVLU College"
goal = "Home"
for name, algorithm in algorithms:
    result = algorithm(start, goal)
    print("=" * 60)
    print(name)
    if result is None:
        print("No path found!")
    else:
        path, cost, explored = result
        print("Path :", " -> ".join(path))
        print("Total Path Cost :", cost, "km")
        print("Number of Nodes Explored :", explored)
```

OUTPUT:-

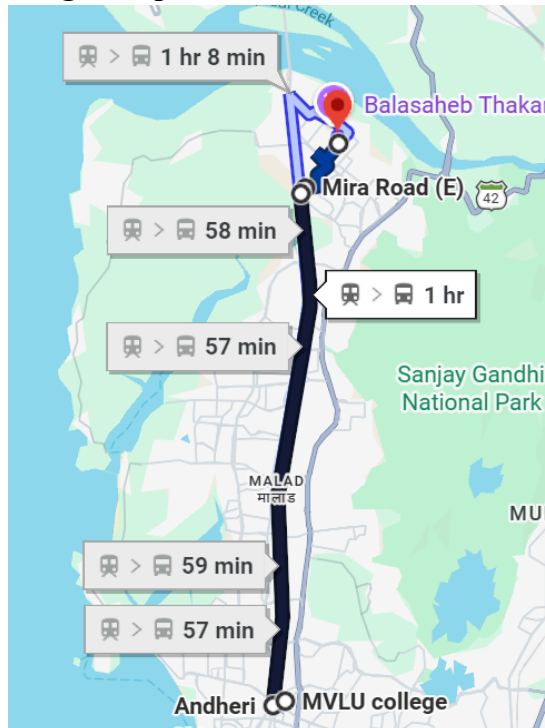
```
=====
Breadth First Search (BFS)
Path : MVLU College -> Andheri Station -> Mira Road Station -> Home
Total Path Cost : 21 km
Number of Nodes Explored : 7
=====
Depth First Search (DFS)
Path : MVLU College -> Andheri -> Dahisar Check Naka -> Mira Road -> Home
Total Path Cost : 23 km
Number of Nodes Explored : 5
=====
Uniform Cost Search (UCS)
Path : MVLU College -> Andheri Station -> Mira Road Station -> Home
Total Path Cost : 21 km
Number of Nodes Explored : 6
=====
Greedy Best First Search
Path : MVLU College -> Andheri Station -> Mira Road Station -> Home
Total Path Cost : 21 km
Number of Nodes Explored : 4
=====
A* Search
Path : MVLU College -> Andheri Station -> Mira Road Station -> Home
Total Path Cost : 21 km
Number of Nodes Explored : 4
```

Additional Task:-**Google Maps Comparison with A* Search:****Google Maps:-****Source:** MVLU College**Destination:** Home (Ramdev Park)**Distance:** Approximately 8.5 km

Estimated Time: 22 minutes

Suggested Route:

MVLU College → Andheri → Mira Road → Home

Google Maps Screenshot:**A* Search:-****Source:** MVLU College**Destination:** Home (Ramdev Park)**Path Found:**

MVLU College → Andheri Station → Mira Road Station → Home

Total Cost (Distance): $1 \text{ km} + 18 \text{ km} + 2 \text{ km} = 21 \text{ km}$ **Total Cost:** 21 km**Nodes Explored:** 4